

CampusPulse — Full-Stack Application Implementation

You are acting as the lead full-stack engineer and software architect for this project.

Build a production-quality full-stack web application called **CampusPulse**.

CampusPulse is a campus issue reporting and resolution platform where students can report problems, confirm that they are also affected by existing problems, track resolution progress, and view issue activity.

Administrators can verify reports, prioritize them, update their status, add resolution notes, and analyze campus issues.

This project is being built as a technical demonstration, so architecture quality, security, maintainability, UI polish, database design, and deployment quality are important.

Do not unnecessarily overengineer the system.

1. PRIMARY STACK

Use:

- Next.js latest stable version
- App Router
- React
- TypeScript with strict mode
- Hono
- Supabase
- PostgreSQL through Supabase
- Supabase Auth
- Supabase Storage
- Supabase Realtime only where genuinely useful
- Zod
- Tailwind CSS
- shadcn/ui
- Docker
- Docker Compose
- Supabase CLI for local Supabase development

Use a single repository.

Do NOT create separate frontend and backend repositories.

2. APPLICATION ARCHITECTURE

The application must use Next.js for the frontend and Hono as the application API layer.

Hono must be integrated into the Next.js application.

All application API routes should exist underneath:

`/api/*`

Implement a Next.js catch-all API route such as:

`app/api/[...route]/route.ts`

and delegate supported HTTP methods to the Hono application.

Conceptually:

Browser | v Next.js | +---- Pages / RSC / Client Components | +---- /api/* | v Hono | +-- authentication middleware +-- authorization middleware +-- Zod validation +-- issue routes +-- admin routes +-- upload routes +-- analytics routes | v Supabase | +-- PostgreSQL +-- Auth +-- Storage +-- optional Realtime

Next.js and Hono must be deployed as ONE application.

Do not create an unnecessary second Node.js server.

3. ARCHITECTURAL RULES

Maintain clear separation between:

UI

API

domain/business logic

database access

authentication

authorization

validation

configuration

Do not put substantial business logic directly inside React components.

Do not put substantial business logic directly inside Hono route definitions.

Use service modules where appropriate.

Use repository/data-access modules where this improves maintainability.

Avoid unnecessary abstractions.

The code should remain understandable to another developer.

4. SUGGESTED PROJECT STRUCTURE

Use approximately:

app/ (public)/ (auth)/ dashboard/ admin/ api/ [...route]]/ route.ts

components/ ui/ layout/ features/ issues/ admin/ auth/

server/ api/ index.ts middleware/ routes/ issues.ts admin.ts auth.ts uploads.ts analytics.ts

```
auth/  
  
services/  
  issue-service.ts  
  confirmation-service.ts  
  admin-service.ts  
  analytics-service.ts  
  attachment-service.ts  
  
repositories/  
  
supabase/  
  server.ts  
  admin.ts
```

lib/ supabase/ browser.ts

```
schemas/  
constants/  
utils/  
types/
```

supabase/ migrations/ seed.sql config.toml

public/

Dockerfile docker-compose.yml .env.example README.md

Modify this structure if there is a technically superior reason.

5. PRODUCT ROLES

Implement two roles.

STUDENT

ADMIN

Store application profile information separately from Supabase auth.users.

Create:

profiles

with:

id UUID primary key referencing auth.users

display_name

role

avatar_url

created_at

updated_at

Roles:

student

admin

Never trust a role supplied by the frontend.

Authorization must always use server/database-derived identity.

6. AUTHENTICATION

Use Supabase Auth.

Implement:

Sign up

Sign in

Sign out

Session persistence

Protected pages

Protected API endpoints

Server-side session resolution

Hono authentication middleware

Use the currently recommended Supabase SSR integration for Next.js.

The authentication session must work correctly with:

Next.js Server Components

Next.js middleware/proxy if needed

Client Components

Hono API requests

Do not store authentication tokens manually in localStorage if Supabase's recommended SSR cookie mechanism should be used instead.

7. DATABASE

Create SQL migrations.

Do not rely on manually creating tables through the Supabase dashboard.

Required tables:

profiles

issues

issue_confirmations

issue_updates

attachments

8. ISSUES TABLE

Create approximately:

id UUID primary key

title

description

category

location

priority

status

created_by

created_at

updated_at

resolved_at

Use appropriate PostgreSQL constraints.

Categories:

network

hardware

software

infrastructure

other

Priority:

low

medium

high

critical

Status:

reported

verified

in_progress

resolved

rejected

Use PostgreSQL enums or equivalent strongly constrained values.

9. ISSUE CONFIRMATIONS

Students should be able to click:

"I'm affected too"

Create:

issue_confirmations

Fields:

issue_id

user_id

created_at

The database MUST enforce:

UNIQUE(issue_id, user_id)

Do not depend only on frontend duplicate prevention.

Users must also be able to remove their confirmation.

Expose confirmation counts efficiently.

10. ISSUE UPDATES / ACTIVITY

Create:

issue_updates

Fields approximately:

id

issue_id

actor_id

event_type

old_value JSONB

new_value JSONB

message

created_at

Use this table to build the issue timeline.

Example events:

issue_created

status_changed

priority_changed

admin_note_added

issue_resolved

issue_reopened

The timeline should appear chronologically on the issue detail page.

11. ATTACHMENTS

Use Supabase Storage.

Create a bucket for issue evidence.

Suggested bucket:

issue-evidence

Create an attachments database table:

id

issue_id

uploaded_by

storage_path

mime_type

size_bytes

created_at

Validate uploads server-side.

Allow images only for MVP.

Set a sensible maximum file size.

Generate safe object paths.

Do not use the original filename as the only storage identifier.

Store storage paths in PostgreSQL rather than treating generated URLs as canonical database identifiers.

12. ROW LEVEL SECURITY

RLS is mandatory.

Enable RLS for application tables.

Create explicit policies.

Students should be able to:

read permitted issues

create issues as themselves

manage only operations explicitly permitted for their own records

create confirmation rows only for themselves

delete only their own confirmation

Students must NOT be able to:

make themselves admin

modify admin-only status fields

change arbitrary issue ownership

modify other users' confirmations

perform administrative actions

Admins can perform required administrative operations.

Do not rely solely on frontend checks.

Do not rely solely on Hono authorization when RLS can provide defense in depth.

13. HONO API

Implement a structured Hono API.

Suggested endpoints:

GET /api/health

GET /api/issues

POST /api/issues

GET /api/issues/:id

PATCH /api/issues/:id

POST /api/issues/:id/confirm

DELETE /api/issues/:id/confirm

GET /api/me/issues

POST /api/issues/:id/attachments

PATCH /api/admin/issues/:id/status

PATCH /api/admin/issues/:id/priority

POST /api/admin/issues/:id/updates

GET /api/admin/analytics

Use RESTful semantics.

Use appropriate HTTP status codes.

14. API RESPONSE FORMAT

Use consistent responses.

Success:

```
{ "data": {}, "meta": {} }
```

Errors:

```
{ "error": { "code": "VALIDATION_ERROR", "message": "...", "details": {} } }
```

Create centralized error handling.

Never return raw PostgreSQL errors to the browser.

15. VALIDATION

Use Zod.

Create shared schemas for:

issue creation

issue updates

filters

pagination

admin status updates

priority updates

upload metadata

environment variables

Validate every mutation server-side.

Frontend validation is additional UX validation and must not replace backend validation.

16. ISSUE FEED

Create a polished issue feed.

Header:

CampusPulse

Search field

Report Issue CTA

Filters

Support filtering by:

status

category

priority

location

Support text search.

Implement pagination.

Do not load the entire database into the browser and filter client-side.

Issue cards should show:

title

category

location

status

priority

confirmation count

created time

optional thumbnail

17. ISSUE DETAIL PAGE

Create:

/issues/[id]

Show:

title

description

category

location

priority

status

reporter

created date

confirmation count

"I'm affected too" button

attachments

activity timeline

If the current user has confirmed the issue, show the appropriate active state and allow removal.

18. CREATE ISSUE

Create:

`/issues/new`

Fields:

Title

Description

Category

Location

Optional image

Use excellent validation UX.

Show field-level validation messages.

Disable duplicate submission.

Show loading state.

Show success feedback.

Redirect to the created issue after successful creation.

19. MY ISSUES

Create:

`/dashboard`

Show:

Issues I reported

Issues I'm affected by

Allow filtering by status.

20. ADMIN DASHBOARD

Create:

/admin

Only admins can access it.

Show:

Total issues

Open issues

Resolved issues

Total confirmations

Issues by category

Issues by status

High-impact unresolved issues

Recent issue activity

Use clean charts only where they genuinely improve understanding.

Do not turn the page into a generic template dashboard.

21. ADMIN ISSUE MANAGEMENT

Admins should be able to:

Verify issue

Change status

Change priority

Add administrative note

Resolve issue

Reopen issue

All important changes must create an issue_updates record.

Implement sensible status transition rules.

Do not permit arbitrary invalid transitions if they can be prevented cleanly.

22. UI DESIGN

The application should look like a modern technical SaaS product.

Visual inspiration:

Linear

Vercel

GitHub

Raycast

Do not clone them.

Use:

clean typography

generous spacing

subtle borders

small-radius or medium-radius cards

excellent hierarchy

responsive layouts

subtle animations

skeleton loading

toast notifications

empty states

error states

confirmation dialogs

accessible forms

keyboard navigation

dark mode

light mode

Use shadcn/ui where appropriate.

Do not overuse cards.

Do not use huge gradients everywhere.

Do not make it look like a generic AI-generated landing page.

23. LANDING PAGE

Create a simple but premium landing page.

Hero:

CampusPulse

"See it. Report it. Solve it."

Supporting text explaining the platform.

CTA:

Report an Issue

Explore Issues

Show useful real statistics when available.

Avoid fake statistics in production.

Include a concise "How it works" section.

24. COMMAND PALETTE

If the core application is complete, add:

Ctrl/Cmd + K

Actions:

Report issue

Search issues

View unresolved issues

Open dashboard

Open admin dashboard when authorized

This is a stretch feature and must not delay core functionality.

25. SUPABASE REALTIME

Realtime is optional.

Only implement it after the core application works.

Potential realtime features:

confirmation count changes

status changes

new issue activity

Do not introduce realtime complexity merely to claim that the application uses realtime.

26. DEVELOPMENT ENVIRONMENT

Create a Docker-based development workflow.

Provide:

Dockerfile

docker-compose.yml

.env.example

README instructions

Use Supabase CLI for the local Supabase environment.

The developer should be able to approximately run:

supabase start

supabase db reset

docker compose up --build

If hot reload inside Docker is reliable, configure it.

Otherwise clearly document the recommended hybrid development approach:

Supabase in Docker

Next.js/Hono running natively

Both workflows may be supported.

27. DOCKERFILE

Create a production-quality multi-stage Dockerfile.

Use stages approximately:

base

dependencies

builder

runner

Use Next.js standalone output.

Runtime image should:

contain minimal required files

run as non-root

use `NODE_ENV=production`

expose port 3000

avoid development dependencies

not contain secrets

Support Docker health checks through:

`/api/health`

where appropriate.

28. DOCKER COMPOSE DEVELOPMENT

Create a development compose configuration.

Do not manually recreate the entire Supabase architecture if Supabase CLI already manages it correctly.

Compose should primarily manage the application development container.

Document how the app reaches local Supabase.

Use named volumes where appropriate.

Do not bind-mount production systems.

29. PRODUCTION ARCHITECTURE

Production should be:

Internet

-> DNS/TLS

-> Reverse proxy

-> CampusPulse Docker container

-> Managed Supabase

Do NOT deploy PostgreSQL inside the CampusPulse production Docker Compose stack.

Use managed Supabase for:

Auth

PostgreSQL

Storage

Realtime if enabled

30. VPS DEPLOYMENT

Provide production deployment instructions for Ubuntu LTS.

Assume Docker Engine and Docker Compose.

Create a production compose configuration if useful.

Application container:

campuspulse

Port internally:

3000

Reverse proxy should expose only:

80

443

Application port should remain private where practical.

Support automatic restart.

Document:

initial deployment

environment setup

migration deployment

image update

rollback

log viewing

health checking

container restart

31. ENVIRONMENT VARIABLES

Create `.env.example`.

At minimum account for browser-safe Supabase configuration and server-only secrets.

Use the current Supabase recommended publishable/secret key naming if applicable.

Never expose service-role/secret credentials through `NEXT_PUBLIC` variables.

Validate required server environment variables at startup.

32. SECURITY

Implement:

server-side authorization

RLS

input validation

safe upload validation

secure authentication cookies

safe error handling

restricted CORS

secure headers where appropriate

database constraints

XSS-safe rendering

audit trail for administrative actions

Never trust:

role from frontend

user ID from frontend when auth identity can determine it

client-provided confirmation counts

client-provided ownership

Do not expose secrets.

33. PERFORMANCE

Implement:

server-side pagination

database indexes

efficient confirmation counts

efficient analytics queries

appropriate Server Components

minimal unnecessary client JavaScript

optimized images

avoid N+1 queries

Do not prematurely add Redis.

PostgreSQL should comfortably handle this application.

34. DATABASE INDEXES

Add sensible indexes such as:

issues(created_at)

issues(status)

issues(category)

issues(priority)

issues(created_by)

issue_confirmations(issue_id)

issue_confirmations(user_id)

issue_updates(issue_id, created_at)

Use indexes based on actual query patterns.

Do not blindly index every column.

35. ANALYTICS

Admin analytics must come from actual database queries.

Include:

total issues

open issues

resolved issues

confirmation volume

issues grouped by category

issues grouped by status

top unresolved issues by confirmation count

Do not hardcode dashboard statistics.

36. TESTING

Implement a pragmatic testing strategy.

At minimum test:

validation

authentication guards

admin authorization

confirmation uniqueness

issue creation

status updates

RLS-sensitive operations

status transition rules

Add an E2E or integration smoke workflow if practical:

student login

create issue

second user confirms

admin changes status

admin resolves

timeline shows changes

37. SEED DATA

Create realistic seed data for development.

Create:

multiple students

an admin profile

issues across categories

different statuses

confirmations

issue updates

Do not put real credentials into the repository.

If authentication users cannot safely be seeded through plain SQL in the intended workflow, document the proper local setup process.

38. DEMO QUALITY

The application will be demonstrated during a technical selection.

Optimize the demo experience.

The evaluator should be able to see:

authentication

student workflow

database persistence

confirmation uniqueness

search/filtering

admin authorization

analytics

status workflow

activity timeline

responsive UI

production deployment

within approximately 5–8 minutes.

39. README

Write a professional README.

Include:

project description

screenshots placeholders if necessary

architecture overview

technology stack

local prerequisites

environment setup

Supabase setup

database migrations

seed process

Docker development

native development

production build

production deployment

testing

API overview

database overview

security model

folder structure

demo accounts setup instructions

Do not put secret credentials in README.

40. CODE QUALITY

Use strict TypeScript.

Avoid `any` unless unavoidable and documented.

Use meaningful names.

Prefer small focused functions.

Avoid giant React components.

Avoid giant Hono route files.

Avoid unnecessary comments explaining obvious code.

Comments should explain architectural decisions or non-obvious behavior.

Keep the system simple enough that a student can explain every major component during an interview.

41. IMPLEMENTATION ORDER

Implement the project incrementally.

PHASE 1

Project foundation

Next.js

Tailwind

shadcn/ui

Hono integration

Supabase clients

environment validation

Docker foundation

/api/health

PHASE 2

Database migrations

profiles

issues

confirmations

updates

attachments

RLS

indexes

seed strategy

PHASE 3

Authentication

sign up

sign in

sign out

protected routes

Hono auth middleware

role authorization

PHASE 4

Issue system

feed

create

detail

search

filters

pagination

PHASE 5

Confirmations

"I'm affected too"

remove confirmation

counts

database uniqueness

PHASE 6

Administration

admin dashboard

analytics

status updates

priority

notes

resolution

timeline

PHASE 7

Storage

evidence upload

file validation

storage policies

display

PHASE 8

Polish

responsive design

dark mode

loading

errors

empty states

accessibility

command palette if time permits

PHASE 9

Testing

security review

RLS verification

production Docker image

deployment documentation

42. IMPORTANT IMPLEMENTATION BEHAVIOR

Do not attempt to generate the entire application as one massive uncontrolled code dump.

First inspect the repository.

Then establish architecture.

Then implement one phase at a time.

After every major phase:

run TypeScript checking

run linting

run relevant tests

build the application

fix errors before continuing

Never knowingly leave TypeScript errors for later.

Never knowingly leave broken imports for later.

Do not create placeholder implementations and call them finished.

43. DATABASE MIGRATION DISCIPLINE

All schema modifications must be represented through migrations.

Never tell me:

"Create this table manually in Supabase."

Generate the migration.

All RLS policies should also exist in migrations.

All indexes should exist in migrations.

Storage configuration/policies should be reproducible where supported.

44. DEFINITION OF DONE

The project is complete only when:

The application builds successfully.

Docker image builds successfully.

Authentication works.

Student authorization works.

Admin authorization works.

RLS is configured.

Issues can be created.

Issues can be browsed.

Search/filtering works.

Confirmations work.

Duplicate confirmations are impossible.

Admin status changes work.

Issue activity is recorded.

Attachments work.

Analytics use real data.

Mobile UI works.

Production environment is documented.

README is complete.

No secrets are committed.

No obvious TypeScript errors exist.

No broken routes exist.

No fake production statistics exist.

45. FIRST TASK

Before implementing application features:

1. Inspect the current repository.
2. Determine the installed package manager and existing dependencies.
3. Propose the final folder structure.
4. Identify any architecture conflicts.
5. Create an implementation checklist.

6. Set up the Next.js + Hono integration.
7. Add /api/health.
8. Configure Supabase client foundations.
9. Create environment validation.
10. Create Docker foundations.
11. Verify the development server and production build.

Then proceed to the database/authentication phase.

Do not introduce Redis, Kafka, Kubernetes, microservices or other infrastructure unless a concrete requirement appears that cannot reasonably be solved by the current architecture.